

Relatório — Atividade 02: Criando um Jogo

Jogo: Defesa do Castelo (Tower Defense Medieval) **Integrantes:** Josué Oliveira e Roberto Rawlison **Plataforma:** HTML5 Canvas + JavaScript puro (módulos ES6, sem frameworks) **Repositório:** github.com/robertorawlison/tower-defense **Data:** junho de 2026

Sobre a escolha do jogo

A atividade sugeria o Tetris, mas optamos por desenvolver um jogo de **Tower Defense** (combinado previamente). A ideia central é defender um castelo de ondas de bárbaros usando apenas torres automáticas: o jogador coleta moedas derrotando inimigos e gasta essas moedas comprando e melhorando torres. São 10 ondas; sobreviver à décima é a vitória.

Apesar de o jogo ser diferente do sugerido, ele cobre os mesmos conceitos pedidos em cada etapa do guia (campo de jogo, controle dos elementos, detecção de colisão, condição de fim de jogo, níveis/dificuldade, pontuação, tela de abertura, attract mode, high scores, persistência, estilos visuais e áudio). A seção 7.2 mapeia cada etapa do guia para o que foi feito no nosso jogo.

7.1. Experiência prévia

Experiência com a linguagem/plataforma (HTML / JavaScript): básica. O grupo já tinha tido algum contato com HTML e JavaScript e entendia conceitos fundamentais (variáveis, funções, manipulação básica de página), mas nenhum dos integrantes tinha experiência sólida com a linguagem nem havia desenvolvido projetos web próprios de porte parecido. Conceitos como módulos ES6, `requestAnimationFrame`, renderização em `<canvas>`, Web Audio API e `localStorage` eram, em boa parte, novos para nós.

Experiência com o domínio do projeto (desenvolvimento de jogos): nenhuma. Esta foi a primeira vez que o grupo desenvolveu um jogo. Não tínhamos familiaridade prévia com conceitos comuns de game dev, como **game loop**, **delta time** (tempo entre quadros para movimento independente de FPS), máquina de estados de tela, detecção de colisão, geração procedural de mapas (labirintos/caminhos), pathfinding (BFS) ou sprites direcionais.

Esse ponto de partida — base em web, zero em jogos — é importante para contextualizar o restante do relatório: boa parte do que foi produzido só foi viável por causa do agente de codificação, e parte das nossas conclusões gira em torno de **quanto entendemos** do que foi gerado.

7.2. Relato do desenvolvimento

Ferramentas e estrutura de agentes

- **Agente de codificação:** Claude Code (CLI/extensão da Anthropic), operando como um único agente com acesso ao sistema de arquivos e ao terminal. Não usamos uma arquitetura de múltiplos agentes — foi um agente principal conduzindo todo o desenvolvimento, capaz de ler/escrever arquivos, rodar comandos (servidor local, git) e verificar o próprio trabalho.
- **Modelos utilizados:** modelos Claude da família 4.x — principalmente **Claude Sonnet** (mais rápido, usado na maior parte das edições) e **Claude Opus** (usado em momentos de planejamento e tarefas mais elaboradas). Trocamos entre eles ao longo do projeto.
- **Geração de imagens:** **ChatGPT** foi usado para gerar as imagens de referência dos sprites (personagens, torres, caminhos, castelo e decorações em estilo pixel art). Nós então recortamos esses sprites e os colocamos na estrutura de pastas `assets/` que o jogo espera.
- **Ambiente:** Windows, com o jogo servido localmente via Python (`python -m http.server` , depois substituído por um `serve.py` próprio — ver Etapa 4). Servir via HTTP é necessário porque módulos ES6 não funcionam abrindo o arquivo direto (`file://`).

Uma característica marcante do fluxo foi o agente **fazer perguntas antes de codar**. Logo no início, em vez de já sair gerando, ele propôs um plano e levantou decisões de projeto (stack, estrutura do mapa, como o jogador perde, número de ondas, etc.), que respondemos. Isso virou um arquivo de plano em Markdown que serviu de contexto para o restante do trabalho.

Etapa 1 — Base do jogo

Começamos com um prompt relativamente amplo descrevendo a ideia, e fomos refinando com prompts complementares. Os principais foram:

"crie um código em javascript [...] um jogo de tower defense no qual vc é um cavaleiro e tem que defender o castelo contra tropas invasoras de bárbaros e seu objetivo é coletar moedas derrotando os bárbaros [...]"

"com essas moedas vc pode comprar torres de vigia com arqueiros, torres com bestas [...] e depois uma catapulta que custa mais caro, são três tiers de armas"

"faça com que ao passar das ondas tenham outros tipos de bárbaros, quero que tenham 3 tipos"

"depois de cada onda o jogador terá uma 'pausa' para poder colocar as torres [...]"

"o mapa será gerado aleatoriamente, ele terá 'caminhos' que os bárbaros vão passar [...] as armas terão pontos fixos para serem colocadas 'ao redor' dos caminhos"

"agora para o plano, faça perguntas e dê sugestões de como pode ser feito"

A partir das nossas respostas às perguntas do agente, fecharam-se as seguintes decisões:

- **Stack:** HTML5 Canvas + JS puro (sem framework).
- **Sem cavaleiro controlável** — apenas torres automáticas.
- **Mapa:** grid com múltiplos caminhos, gerado proceduralmente.
- **Visual:** sprites PNG externos, com *fallback* de retângulos coloridos caso algum sprite não carregue (o que permitiu testar a mecânica antes de ter os sprites prontos).

- **Derrota:** castelo com HP (100). **Vitória:** sobreviver às 10 ondas.
- **Upgrade in-place** das torres.
- **Grid 36×21 células de 48px** (canvas 1728×1008).
- **Mapeamento dos inimigos:** martelo = básico, furioso = veloz, machado = tanque.

O resultado da etapa foi a base jogável: geração de mapa com caminhos, autotile dos tiles de caminho, castelo com barra de HP, as 3 torres (arqueiro, besta, catapulta) com upgrade, os 3 tipos de bárbaro, projéteis (incluindo o tiro parabólico da catapulta), as 10 ondas, a fase de construção entre ondas e as condições de vitória/derrota. O código foi organizado em módulos (`main.js`, `config.js`, `map.js`, `maze.js`, `tower.js`, `enemy.js`, `projectile.js`, `wave.js`, `ui.js`, `render.js`, etc.), o que facilitou as alterações posteriores.

Mapeamento com o guia (Etapa 1 do PDF): o "campo de jogo" é o mapa em grid; o controle das peças vira o posicionamento/upgrade de torres; "detecção de colisão" aparece no acerto dos projéteis e na chegada dos inimigos ao castelo; a "condição de fim de jogo" é o HP do castelo zerar; e a informação da "próxima peça" corresponde à indicação da próxima onda.

Etapa 2 — Níveis (dificuldade) e pontuação

No guia, esta etapa é sobre subir de nível e pontuação. No nosso jogo, ela se traduziu em:

- **Níveis de dificuldade (Fácil / Difícil):** uma tela de seleção no início, que muda a

estrutura e o número de caminhos pelos quais os bárbaros chegam (no fácil, 2 caminhos que atravessam 2 quadrantes cada; no difícil, 4 caminhos, um por quadrante). Também foi adicionado um botão para **regenerar o labirinto**.

- **Pontuação:** decidimos pontuar com base na **vida do castelo preservada ao final**. Proteger o castelo 100% (sem perder nenhum ponto de vida) dá pontuação máxima (100%); abaixo disso, a pontuação cai proporcionalmente, com "medalhas" por faixa (Perfeito, Ouro, Prata, Bronze, Sobreviveu) e Derrota = 0%. A pontuação é exibida na tela de fim de jogo.

- **High scores (opcional do guia):** criamos um **ranking das melhores tentativas**, que

ordena as partidas por vitória, pontuação e onda alcançada, destacando a tentativa atual e sinalizando quando é um novo recorde.

Prompt representativo desta etapa:

"crie uma espécie de ranking de qual tentativa foi a de maior sucesso e que tenha uma pontuação no final, como sendo de 100% caso você tenha protegido todo o castelo sem cair nenhuma vida, e outras pontuações com base em quanto de vida o castelo ficou"

Etapa 3 — Tela de abertura, game over e loop de arcade

Esta etapa seguiu de perto o guia, inclusive os itens opcionais:

- **Tela de abertura:** o jogo abre numa tela de título e aguarda o jogador interagir (qualquer tecla ou clique) para começar.
- **Game over / volta ao início:** ao fim da partida (vitória ou derrota), é exibido o resultado com a pontuação e, em seguida, o jogo retorna à tela de abertura.
- **Attract mode (opcional):** depois de um tempo parado na tela de abertura, o jogo entra em modo demonstração, em que um "bot" joga sozinho (constrói torres, dá upgrade e inicia ondas). A demonstração termina assim que o jogador interage.
- **Tela de high scores no loop (opcional):** implementamos o loop clássico de arcade — tela de

abertura → (após um tempo) attract mode → (após mais um tempo) tela de high scores → volta para a abertura, repetindo.

Prompt representativo (resumido) desta etapa:

"o jogo deve começar com uma tela de abertura e esperar o usuário interagir para começar. Ao fim de uma jogada (game over) [...] o jogo deve retornar para a tela de abertura. [...] attract mode mostrando uma demonstração [...] combinado com a tela de high scores no loop tradicional dos jogos de arcade [...]"

Etapa 4 — Outras adições

Implementamos várias das sugestões da quarta etapa do guia, além de extras próprios:

- **Persistência das pontuações (6.1):** o ranking é gravado no `localStorage` do navegador, de modo que as melhores pontuações continuam disponíveis ao reabrir o jogo.
- **Suporte a estilos visuais (6.2):** o jogo usa **sprites PNG** para grama, caminhos, decorações, castelo, torres e inimigos (com renderização direcional — 4 direções — para torres e bárbaros), mantendo o **fallback** de formas coloridas. Os sprites foram gerados com ChatGPT e recortados por nós.

- **Efeitos sonoros e música (6.4):** adicionamos **música de fundo** (uma melodia medieval

simples sintetizada via Web Audio API) e vários **efeitos sonoros em MP3**, disparados em eventos específicos: matar um bárbaro, início de onda ("os bárbaros atacam"), início de partida no modo fácil/difícil, conclusão de onda, construção/upgrade de torre, dano ao castelo e os jingles de vitória e derrota. Há um botão de mudo (e atalho **M**), e o áudio respeita a regra dos navegadores de só iniciar após a primeira interação do usuário.

- **Efeitos visuais (6.5):** indicadores pulsantes que mostram, na fase de preparação, ****de onde os bárbaros vão surgir**** (círculo + seta + rótulo), e barra de HP destacada sobre o castelo.

- **Outras adições (6.6):**

- **Modo de velocidade** (1x / 2x / 3x / 4x) para quem quer jogar mais rápido.
- Ajuste fino da **geração dos caminhos** para penalizar curvas em excesso (deixar os caminhos menos "zig-zag" e mais retos).
- **Servidor local sem cache** (`serve.py`): durante o desenvolvimento, tivemos problemas recorrentes de cache do navegador (arquivos novos e antigos se misturavam e quebravam o jogo). Criamos um pequeno servidor Python que envia cabeçalhos `no-cache`, resolvendo o problema de vez.

Retrabalho e dificuldades ao longo do processo

Como pede o guia, vale registrar que **algumas etapas exigiram retrabalho** e vários prompts auxiliares de correção. Os casos mais relevantes:

- **Cache do navegador:** o sintoma mais confuso do projeto. Depois de algumas mudanças, os botões "paravam de funcionar". A causa não era o código, e sim o navegador servindo uma mistura de arquivos novos e em cache (um erro do tipo `**módulo não exporta tal função**`). Resolver isso exigiu diagnosticar com um detector de erros na tela e, por fim, trocar o servidor por um sem cache.
- **Tamanho do mapa:** pedimos para diminuir o mapa, mas isso fez alguns caminhos serem gerados "para fora" da área visível. Tivemos que reverter essa mudança.
- **Calibragem dos caminhos:** o ajuste do "zig-zag" foi iterativo — pedimos para reduzir as curvas várias vezes em sequência até chegar no resultado desejado.
- **Frestas entre os tiles:** apareceram linhas verdes claras entre os quadrados do mapa (artefato de renderização), que exigiram alguns ajustes de desenho dos tiles.

Em geral, foram correções pontuais e o agente conseguiu diagnosticar e resolver a maioria a partir da descrição do sintoma (e, quando preciso, de um print de tela).

7.3. Conclusões e comentários

Sobre a falta de experiência e seu impacto

A falta de experiência com desenvolvimento de jogos **não impediu** o projeto de avançar — pelo contrário, o agente foi capaz de produzir uma base funcional e ir incrementando recurso sobre recurso. O conhecimento básico de web ajudou a acompanhar a estrutura geral (HTML, arquivos, ideia de funções), mas conceitos específicos de jogos (game loop, delta time, geração procedural, máquina de estados de telas) chegaram prontos pelo agente, sem precisarmos estudá-los a fundo de antemão.

Por outro lado, **a falta de experiência apareceu na hora de avaliar e direcionar**. Em vários momentos só conseguíamos perceber que algo estava errado pelo resultado visual ("o caminho está bugado", "as linhas verdes", "o botão não funciona") e dependíamos do agente para entender a causa. Se fôssemos **continuar** o projeto de forma séria, sim, seria necessário estudar mais — tanto a linguagem/plataforma quanto fundamentos de desenvolvimento de jogos — para conseguir tomar decisões técnicas com mais autonomia e revisar o que é gerado.

O que aprendemos e se faríamos sem agentes

Aprendemos bastante sobre **como um jogo é estruturado** na prática: a ideia do game loop rodando a cada quadro, a separação em módulos (mapa, inimigos, torres, UI, render, áudio), o uso de `localStorage` para persistência, a Web Audio API para som, e o porquê de um jogo em módulos ES6 precisar ser servido por HTTP. Também aprendemos, na marra, sobre cache de navegador.

Conseguiríamos fazer um projeto parecido **sem** agentes? Honestamente, não nesse prazo nem nesse nível de acabamento, dado o nosso ponto de partida (zero em game dev). Seria possível com bastante estudo e tempo, mas o agente comprimiu enormemente a curva de aprendizado e o esforço de implementação.

Sobre a legibilidade e manutenção do código

Olhando o código final, a impressão é de que o resultado é **razoavelmente legível e organizado**: os arquivos têm responsabilidades separadas, os nomes são em geral claros e as configurações ajustáveis (custos, HP, velocidades, ondas) ficam centralizadas em `config.js`, o que facilita mexer no balanceamento. Isso é positivo para manutenção.

Ainda assim, achamos que **alguma refatoração seria saudável** se o projeto crescesse — por exemplo, o `main.js` concentra bastante responsabilidade (loop, estados de tela, renderização, áudio), e poderia ser dividido. Para o escopo de uma atividade, no entanto, o nível de organização nos pareceu adequado.

Sobre o déficit de compreensão (*comprehension debt*)

Este é o ponto mais honesto a registrar. **Entendemos o projeto em nível geral** — sabemos o que cada arquivo faz, conseguimos localizar onde mexer para mudar um valor, um som ou um texto, e acompanhamos a lógica de alto nível. Mas **não entendemos em profundidade** todas as partes, especialmente as mais "algorítmicas", como a geração procedural do labirinto (DFS/backtracking, o viés de direção que controla as curvas) e o BFS que resolve os caminhos.

Se fosse necessário fazer **modificações pequenas** (trocar sons, valores, textos, cores, ajustar balanceamento, adicionar um botão), provavelmente conseguiríamos mexer sozinhos, com algum esforço. Já modificações **estruturais** (mudar o algoritmo de geração de mapa, reescrever o sistema de pathfinding, alterar a arquitetura de telas) seriam difíceis sem o apoio do agente.

Sobre **como julgamos se uma modificação pedida deu certo**: na prática, julgamos principalmente **pelo comportamento observável** — rodando o jogo e vendo se faz o esperado (o som toca, a tela aparece, o caminho fica mais reto, o botão responde). Esse é um critério funcional, mas limitado: ele não garante que o código por trás esteja correto, eficiente ou bem escrito — só que "parece funcionar". Reconhecer essa limitação é, em si, um dos aprendizados da atividade: a facilidade de gerar muito código rapidamente vem acompanhada do risco de acumular partes que funcionam, mas que não compreendemos totalmente.

Comentários finais

No geral, a experiência de desenvolver com um agente de codificação foi **muito positiva e produtiva**. O fluxo de "conversar, pedir, ver rodar e ajustar" é rápido e motivador, e o agente foi bom tanto em construir quanto em diagnosticar problemas a partir de descrições informais (e até de prints). Os pontos de atenção que levamos da atividade são: (1) a importância de ainda estudar os fundamentos para não ficar totalmente dependente; (2) o cuidado com o déficit de compreensão; e (3) o fato de que "funciona na tela" não é o mesmo que "o código está bom" — validar de verdade exige entender o que foi feito.